MFM - Project 1 Report

Part 1: Pretrained ViT-B/16 | Part 2: CLIP Image–Text Matching | Part 3: CLIP and "On" vs. "Under"

Part 1 — Understanding a Pretrained Vision Transformer

For this part I loaded torchvision's **vit_b_16** with the ImageNet-1k weights (86.6M parameters, 1000 output classes) — nothing was trained here, I just used the checkpoint as-is. I put together a small collection of 20 images from Wikimedia Commons (all CC0 / CC BY / CC BY-SA): 5 categories × 4 images each — golden retriever, zebra, school bus, banana, and acoustic guitar. I picked categories that each map to exactly one ImageNet class, and the class indices are looked up from the checkpoint's own label list instead of typed by hand. Every image goes through the checkpoint's own preprocessing (resize to 256, center crop to 224, ImageNet normalization).

Task 1 — Image Patches and Model Inputs

I wrote a **patchify** function that cuts a (C, H, W) image into non-overlapping 16×16 squares in row-major order (left to right, top to bottom), which is the same order the ViT uses for its token sequence. I also wrote **unpatchify** to put the patches back together, as a sanity check — cutting an image up and reassembling it has to give back the exact original.

What happened

- A 224×224 image with 16×16 patches gives 224 / 16 = 14 patches per side, so 14 × 14 = **196 patches**. The model adds one CLS token in front, so the sequence length is **197 tokens**. I checked this against the actual model, not just my arithmetic: the patch tokens come out as (1, 196, 768), the positional embedding is (1, 197, 768), and a hook on the first encoder block confirmed (1, 197, 768) goes in.
- **Patch projection:** each 16×16×3 patch (768 raw numbers) gets turned into one 768-dim token by the *same* learned linear map. torchvision implements it as a convolution with kernel 16 and stride 16, which is mathematically the same thing. To prove it, I took my own patches, flattened them, and multiplied by the pretrained weights as a plain matrix product — it matched the model's patch tokens to within 2.26×10^{-6}.
- **Positional embeddings:** self-attention treats its input as an unordered set, so without position info the model can't tell the top-left patch from the bottom-right one. Each of the 197 positions gets a learned vector *added* to its token. When I plotted cosine similarity between position embeddings on the 14×14 grid, each position was most similar to positions in its own row and column (a cross shape) — so the model learned the 2-D grid layout on its own, even though it was only ever given a 1-D index.
- **CLS token:** one extra learned vector at position 0 that doesn't correspond to any part of the image. Through attention it can pull in info from every patch across the 12 layers, and its final state is the single vector that goes into the classification head.

My patch picture vs. the learned projection

- My patch visualization is *just a rearrangement of pixels* — patchify has no parameters, and the tiles still look like the photo because each one literally is a piece of the photo.

- The model's patch projection is a different kind of thing: same grid, but each flattened patch gets multiplied by a learned 768×768 weight matrix plus a bias, giving a feature vector that isn't an image and can't really be viewed as one.
- Each row of that weight matrix can be reshaped to 3×16×16 and shown as a little RGB "filter" — those are what the model learned to look for in a patch (edges, colors, frequency patterns). They're fixed properties of the checkpoint, the same for every image, while my patch grid changes with every image.

Task 2 — Predictions on a Small Image Collection

I ran all 20 images through the model in batches of 4 under torch.no_grad() and saved the full 20 × 1000 probability matrix. Since the model chooses among 1000 classes and not just my 5, a wrong prediction can land on a class outside my categories, so the confusion matrix gets an extra "other ImageNet class" column.

Category	ImageNet index	Top-1 correct	Top-5 correct
Golden retriever	207	4 / 4	4 / 4
Zebra	340	4 / 4	4 / 4
School bus	779	4 / 4	4 / 4
Banana	954	4 / 4	4 / 4
Acoustic guitar	402	3 / 4	4 / 4
Overall	—	**19 / 20 (95%)**	**20 / 20 (100%)**

The interesting cases

Image	Top predictions	Result
acoustic_guitar_4	electric guitar 0.53, acoustic guitar 0.37	Wrong (true label ranked 2nd)
golden_retriever_4	golden retriever 0.44, kuvasz 0.27, Labrador 0.07	Correct but uncertain
banana_3	banana 0.75, barrow 0.09	Correct, mildly uncertain
school_bus_4	school bus 0.90, garbage truck < 0.002	Correct, most confident

Takeaways

- The only miss was **acoustic_guitar_4**, a teal three-quarter-size acoustic guitar that got called an electric guitar. After the center crop the model sees mostly the neck and upper body, and the solid bright color plus the cutaway shape look a lot more like an electric guitar than the natural-wood guitars in the other three photos. It's a near miss between two closely related classes.
- **golden_retriever_4** is a very pale, cream-colored dog shown head-and-shoulders only. The runners-up (kuvasz, Great Pyrenees) are both white-coated breeds, so the model's doubt basically follows the coat color. **banana_3** shows bananas on a tray on a roadside stand, and

"barrow" (a cart) is honestly a fair description of that stand — the model just has to pick one label.

- Kind of a weird one: 17 of the 20 top-1 probabilities sit between 0.83 and 0.91, and none go above 0.91 even for super clear images. That ceiling looks like what you get from training with label smoothing and mixup/cutmix, since the model never gets trained toward a target of 1.0. I didn't train this checkpoint, so that's my interpretation, not something I measured — but it means "0.90" here really reads as "about as sure as this model gets."
- With only 4 images per category, one image moves accuracy by 5 percentage points, so these numbers describe this collection only.

How the fixed output categories limit the questions I can ask

- **Only 1000 answers exist.** I picked my 5 categories *because* they're in the label list. Something like "sunflower" or "person" can never be predicted.
- **Exactly one label, probabilities sum to 1.** The model can't say "both" (banana_3) or "none of these" — an unknown object still gets a confident-looking distribution.
- **The granularity is fixed.** ImageNet has ~120 dog breeds, so golden_retriever_4 becomes a hard breed-ID problem, but there's no way to just ask "is this a dog?" without summing probabilities myself. And I can't go finer than a listed class, like "a *teal* acoustic guitar."
- **Only "what's the main object?"** Questions about color, counts, actions, or spatial relationships have no outputs at all, and changing the question means retraining a new output layer.

Part 2 — CLIP for Image–Text Matching and Zero-Shot Classification

Here I switched to OpenAI's **CLIP ViT-B/32** (openai/clip-vit-base-patch32 through Hugging Face), with the weights frozen the whole time, and used the same 20 images from Part 1. The encoders give back raw, un-normalized 512-dim embeddings; everything after that — normalization, similarity, logits, softmax, and the prediction — I wrote by hand.

Task 1 — Implementing the Similarity Calculation

The four steps:

1. L2-normalize every embedding to length 1
2. Cosine similarity matrix: $S = I_{norm} \cdot T_{norm}^{T}$ (for unit vectors, cosine is just the dot product)
3. Logits = exp(logit_scale) × S (the learned scale is 100.0 for this checkpoint)
4. Softmax over the candidate texts in each row, then argmax picks the category

My logits matched the model's built-in logits_per_image to within 1.53×10^{-5} (on values from 11.90 to 33.85), and the predicted category was identical for all 20 images.

Cosine vs. logits vs. softmax

- **Cosine similarity** is the raw geometric number, and it lives in a surprisingly narrow band: 0.119 to 0.338 over all 100 image–text pairs. Matching pairs average 0.308 and non-matching ones 0.187, so a "match" and a "non-match" only differ by about 0.12. A cosine of 0.31 is a strong match for CLIP even though it'd be weak almost anywhere else.

- **Logits** multiply that by 100, which doesn't change the ranking at all — it just stretches the narrow band so softmax can be decisive. A 0.12 cosine gap becomes a 12-point logit gap.
- **Softmax** exponentiates, so a logit gap of 12 turns into a probability ratio of about $e^{12} \approx 160{,}000$. That's why the probability heatmap is almost all 0s and 1s while the cosine heatmap only shows moderate contrast.

Why the probabilities depend on the candidate list

I scored one image (golden_retriever_4) against three different candidate lists. The cosine and logit for "a photo of a golden retriever" only depend on that image and that text, so they're identical every time — only the probability moves:

Candidate set	Cosine	Logit	p(golden retriever)
A: the 5 project categories	0.3092	30.92	1.000
B: only "golden retriever" vs. "zebra"	0.3092	30.92	1.000
C: 5 categories + "labrador retriever" + "kuvasz"	0.3092	30.92	0.705 (labrador 0.241, kuvasz 0.054)

The image didn't change and the model didn't get less sure of anything — the *question* changed. Softmax divides by the sum over every candidate in the list, so a CLIP probability means "the best of these options," not "the probability this is a golden retriever." A list with no right answer would still sum to 1.

Task 2 — Comparing Prompt Wording

I tried three templates without touching the weights: the bare category name, "a photo of a {category}," and "an image showing a {category}" (with "an" where it's grammatical). The logit margin is the logit of the true text minus the best wrong text — it's more informative than probability here since the probabilities are all saturated near 1. As a supplementary check I also added one look-alike distractor per category (labrador retriever, horse, city bus, plantain, electric guitar) for 10 candidates total.

Template	5-way accuracy	Min p(true)	Mean cos to true text	Mean logit margin	10-candidate accuracy
{category}	20 / 20	0.9987	0.2963	9.64	17 / 20
a photo of a {category}	20 / 20	0.9918	0.3081	9.96	16 / 20
an image showing a {category}	20 / 20	0.9968	0.3145	10.48	18 / 20

What I noticed

- On the required 5-category task, all three templates got 20/20, so the confusion matrices are identical and perfectly diagonal. Wording didn't change a single prediction — but it did change the scores.

- The effect isn't uniform. **zebra_4** gains steadily as the wording gets more descriptive (margin 9.20 → 11.24 → 12.62), and school buses do the same (category mean 9.89 → 10.44 → 11.86).
- Bananas go the other way for "a photo of a…": the category margin drops from 8.00 to 5.92, then recovers to 7.70. For banana_1, the banana logit barely moves (27.3 → 27.5) but the "school bus" logit jumps from 18.7 to 21.8 — the margin shrank because a competitor got stronger, not because the right answer got weaker.
- None of this shows up in the softmax probabilities, which all stay above 0.99. The probability is basically a saturated summary of the margin.
- With the 10 harder candidates, wording *does* flip predictions: banana_2 and banana_3 switch between "banana" and "plantain" depending on the template. Every error in that check is a banana called a plantain. That said, 1–2 images out of 20 is way too small to rank the templates, and banana vs. plantain is honestly hard to judge by eye for the green bunches.

Why changing the prompt isn't fine-tuning

The model stays in eval() mode, every forward pass runs under torch.no_grad(), and there's no optimizer — not one parameter changes. Changing the prompt just changes the *input* to the frozen text encoder, which lands on a different point in embedding space that the (fixed, computed-once) image embeddings get compared against. Fine-tuning would change the *function* itself by updating weights from labeled examples. So prompting is free and instantly reversible, but it can only pick from representations the model already has — it can't teach CLIP something it never learned, which is what Part 3 pokes at.

Comparison with Part 1

Model	Candidates	Correct
ViT-B/16 (ImageNet)	1000 ImageNet classes	19 / 20
ViT-B/16, restricted to my 5 classes	5 classes	20 / 20
CLIP ViT-B/32 (all three templates)	5 texts	20 / 20
CLIP ViT-B/32 (supplementary)	10 texts incl. look-alikes	16–18 / 20

- The only difference between the models was acoustic_guitar_4, which the ViT called "electric guitar." But this isn't a fair head-to-head: the ViT had to choose among 1000 classes that include electric guitar and kuvasz, while CLIP only got five unrelated options. Restrict the ViT to the same 5 classes and it also gets 20/20; give CLIP look-alikes and it drops to 16–18/20. The candidate set explains more of the gap than the model does.
- They're also trained differently (supervised cross-entropy on fixed labels vs. contrastive matching with ~400M web image–text pairs), use different patch sizes (16 vs. 32), and their probabilities aren't comparable — the ViT's 0.89 is a share of 1000 options, CLIP's 0.999 is a share of 5 after a ×100 scale.

Task 3 — Exploring the Representation Space

Next I computed image–image cosine similarity between the normalized CLIP image embeddings, ordered by category so each category forms a 4×4 block on the diagonal of the heatmap.

Pairs	Count	Mean cosine	Min	Max
Within a category	30	0.838	0.676	0.951
Across categories	160	0.505	0.347	0.661

Category	Within-category mean	Closest other category
Zebra	0.923	Golden retriever (0.595)
School bus	0.864	Zebra (0.510)
Acoustic guitar	0.831	Golden retriever (0.532)
Golden retriever	0.797	Zebra (0.595)
Banana	0.774	Zebra (0.556)

What I found

- The ranges don't overlap at all — the least similar same-category pair (0.676) still beats the most similar cross-category pair (0.661), and every image's nearest neighbor is from its own category (20/20). Note the baseline is high though: two totally unrelated photos still score around 0.5.
- Zebras are the tightest group; zebra_1–zebra_3 is the most similar pair overall (0.951) — both side-on, full-body shots on open savanna. Bananas are the loosest (0.774): banana_2 is a studio-style shot of one yellow bunch with stickers, and it only scores 0.68–0.70 against the green roadside-stall photos.
- Golden retrievers and zebras are the closest pair of categories, and the top three cross-category pairs all involve golden_retriever_4 with a zebra. Both are animals outdoors, and the top pair (0.661) even matches in composition — a close-up of an animal's head against blurry vegetation.
- Some less obvious ones: golden_retriever_2 (indoors, warm orange-brown tones, wooden floor) is closer to the wood-colored indoor guitars than the other dogs are, and acoustic_guitar_3 (the only guitar photographed outdoors on grass) is the guitar closest to the school buses. Color and setting seem to matter.
- Overall: the main object dominates (moving similarity by about 0.33), but underneath that CLIP also encodes broad semantics (animals near animals), scene/background, color palette, and composition, each moving similarity by roughly 0.1–0.2. Makes sense since captions describe the whole picture, not just the labeled object. These attributions are my reading of 20 images, though — nothing was varied in a controlled way, which is what Part 3 does.

Part 3 — A Controlled Study of CLIP's Limitations: "On" vs. "Under"

The question here: can CLIP ViT-B/32 tell *where* an object is relative to another object, when both descriptions name the exact same two objects? E.g. "A mug **on** a table" vs. "A mug **under** a table." Same frozen checkpoint and the same hand-written similarity code as Part 2.

Task 1 — Hypothesis and Comparison Sets

Hypothesis (written before running anything)

CLIP is trained to match whole images with whole captions, and Part 2 showed its embedding is dominated by which objects are present. Usually a caption can be matched to its image from the nouns alone, so the training gives little pressure to get prepositions right.

- **H1:** When two descriptions have the same objects and differ only in "on" vs. "under," CLIP will pick the right description close to chance (50%), and will get *both* images in a set right in clearly fewer than half the sets.
- **Supporting prediction:** the two descriptions of a set will have very similar text embeddings (cosine close to 1), much more similar than descriptions of different objects.

Data

The images come from **What'sUp, Controlled Images A** (Kamath, Hessel & Chang, EMNLP 2023; MIT license), where household objects were photographed in fixed positions relative to a table, in one room, from one camera position. I fixed the selection rules in a script before looking at any model output: only photos where the reference object is the table, only the "on" and "under" relations, *every* object the dataset has in both positions (nothing hand-picked or dropped), and the dataset's own captions unedited. That gave **27 comparison sets** (54 images, 54 captions). In each set, the on-image should match "A <object> on a table" and the under-image should match "A <object> under a table" — since both captions name the object and the table, recognizing objects can't solve it; only the relation can.

Possible confounds

- **Position in the frame:** "on" objects are always higher in the image and "under" objects lower, so a model could succeed just by learning "object high up" (this would inflate accuracy, not hurt it).
- **Background and shadow:** on the table the object is against the wall; under it, it's against the floor and in shadow, so darker and lower contrast.
- **Object size:** after resizing to 224 px with 32-px patches, a lemon or knife covers about one patch.
- **Plausibility prior:** objects on tables are way more common in captions than objects under tables, so the model might just prefer "on." I report on/under images separately to catch this.
- **Other stuff:** a few captions are ungrammatical ("A orange," "A scissors") but kept as-is so both captions in a set stay parallel; it's all one room/table/photographer; and the sets aren't statistically independent.

Task 2 — Evaluating and Visualizing

For each set I built a 2×2 cosine similarity matrix (rows: on/under images, columns: on/under captions), with the correct matches on the diagonal. For each image, CLIP "picks" whichever caption scores higher. The learned ×100 scale multiplies all four entries equally, so it can't change a decision — I just compared cosines directly. The **margin** is the cosine to the correct caption minus the cosine to the wrong one.

Metric	Result
Image-to-description accuracy	35 / 54 (64.8%), exact binomial p = 0.040 vs. 50%
“On” images correct / “under” images correct	20 / 27 and 15 / 27
Sets with both images correct	8 / 27 (29.6%) — coin flips would give 25%
Images assigned the “on” caption	32 / 54
Mean margin (mean |margin|)	+0.0045 (0.0096), range −0.013 to +0.040

And here’s how similar the captions themselves are to the text encoder:

Caption pairs	Mean cosine	Range
Same object, different relation (the two captions of a set)	0.926	0.847 – 0.951
Different object, same relation	0.804	0.611 – 0.951
Different object, different relation	0.740	0.561 – 0.896

What happened

- 64.8% is above chance, but not by a lot — p = 0.040, and even that’s optimistic since it treats the 54 decisions as independent.
- No set had both images wrong; the other 19 sets each got exactly one right. Within a 2×2, getting exactly one right means the model picked the *same caption for both images*. In 12 sets it said “on” for both (banana, book, dog, lemon, pillow, …) and in 7 it said “under” for both (bowl, cup, knife, mug, …). So in 19 of 27 sets, the chosen caption didn’t depend on where the object actually was — it depended on the object.
- The margins are tiny. The mean margin is +0.0045, about **a tenth** of the ~0.12 gap that separated object categories in Part 2. But the images still match their captions fine in absolute terms (mean cosine 0.291, close to Part 2’s 0.31), so CLIP clearly recognizes “an object and a table” — it just barely registers which relation holds.
- Which photo it is matters more than the relation: in the book set, the “under” photo scores higher than the “on” photo against *both* captions (0.314 / 0.304 vs. 0.293 / 0.282). The photo-to-photo difference is ~0.02, the caption-to-caption difference only ~0.01.
- The supporting prediction holds: swapping the preposition moves a caption way less than swapping the object (1 − 0.926 vs. 1 − 0.804). The text encoder isn’t totally blind to the preposition though — sharing a relation makes otherwise different captions a bit more similar (0.804 vs. 0.740).
- Failures include big, obvious objects like the pillow, scarf, and dog, so small object size can’t be the whole explanation.

Verdict on H1

Partly supported. Like I predicted, both images were matched in far fewer than half the sets (8/27), and the captions are nearly interchangeable to the text encoder. But per-image accuracy (64.8%) is a little better than "close to chance," so there's a weak signal about the relation — just too weak to match a pair reliably.

Task 3 — Testing One Explanation

The explanation: maybe the problem is on the text side. "On" and "under" are short, super common words that show up in captions in lots of non-spatial senses ("a book on dinosaurs," "under construction"), so maybe the text encoder barely separates them. If so, more explicit spatial wording should help.

The one change: "on" → "**on top of**" and "under" → "**underneath**" in every caption (e.g. "A mug on top of a table" / "A mug underneath a table"). Everything else — the 54 images and their cached embeddings, the model, preprocessing, the rest of each caption, the set pairings, and the metrics — stayed exactly the same. If the explanation is right, the two captions of a set should get less similar and accuracy should go up clearly.

Metric	Baseline (on / under)	Changed (on top of / underneath)
Image-to-description accuracy	35 / 54 (64.8%)	34 / 54 (63.0%)
"On" images correct	20 / 27	14 / 27
"Under" images correct	15 / 27	20 / 27
Sets with both images correct	8 / 27	8 / 27
Images assigned the "on" caption	32 / 54	21 / 54
Mean margin	+0.0045	+0.0028
Mean cosine between a set's two captions	0.926	0.915
p-value vs. 50%	0.040	0.076

What happened

- Basically no net effect: 64.8% → 63.0%, and still 8/27 sets fully correct (only cap and spatula were fully correct both times).
- But the aggregate hides a lot of movement. 25 of the 54 decisions flipped — 12 got fixed and 13 got broken (sign test p = 1.0). What really changed was the *direction of the bias*: the model went from favoring "on" (32/54) to favoring "underneath" (only 21/54 assigned "on"). So "on" images got worse and "under" images got better.
- The first half of the prediction sort of happened (the two captions got slightly less similar, 0.926 → 0.915), but the second half didn't — accuracy didn't improve. So "the short prepositions are too weak a cue" is **not supported** as the main cause. Rewording shifted which caption is preferred overall without making that preference follow the image.

Supplementary check: is the relation in the image embeddings at all?

This one uses no text, just the cached image embeddings. For each set, I estimated a "relation direction" from the *other* 26 sets as the average of (on-image − under-image), then projected the held-out set's two images onto it and called the higher one "on."

- Image embeddings alone ordered the held-out set correctly in **27 of 27** sets ($p = 1.5\times10^{-8}$). So the image encoder *does* produce systematically different embeddings for on vs. under photos.
- The text encoder also has a consistent "on minus under" direction (mean pairwise cosine 0.875 across sets). But the image direction and the text direction barely line up — their cosine averages only +0.09 (range −0.01 to +0.22). So the information exists on both sides, it's just not aligned across the two modalities, which fits a contrastive objective that rarely needs the relation to tell a caption's image apart from other images.

Alternative explanations and limits

- The image-side signal might not be "the relation" at all — "on" also always means "higher in the frame, against the wall." The 27/27 shows the photos are separable, not that CLIP understands the object–table relationship.
- I only tested one rewording. Other phrasings ("sitting on," "on the floor beneath," or "a photo of…" templates) might act differently. "On top of" also adds tokens and "underneath" is a rarer word, so length and word frequency changed along with explicitness.
- 27 sets from one room, one table, one camera, and one checkpoint (ViT-B/32). Larger CLIP models or natural photos might behave differently, and the baseline is only marginally above chance with the two conditions differing by one image — neither supports a strong claim.

Conclusion

In this controlled setting, CLIP ViT-B/32 reliably recognizes that an image shows a given object and a table, but tells "on" from "under" only a bit better than chance (64.8% of images, 8 of 27 sets fully correct), with margins about a tenth the size of the ones that separate object categories. Making the wording more explicit didn't help (63.0%, 8 of 27); it just flipped which caption the model favors. The evidence points away from a pure wording problem and toward weak alignment between how the relation is encoded in images vs. in text.